



In-Context Learning (ICL)



Mohammad Javad Dousti

- * Mohit Iyer's advanced NLP's slides from UMass.
- * Matt Gormley slides for CMU's Generative AI course.

- * Mohit Iyer's advanced NLP's slides from UMass.
- * Matt Gormley slides for CMU's Generative AI course.

Few-shot Learning with LLMs

- Suppose you have...
 - a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ and N is rather small (i.e. few-shot setting)
 - a very large (billions of parameters) pre-trained language model
- There are two ways to “learn”:

Our focus today



Option A: Supervised fine-tuning

- **Definition:** fine-tune the LLM on the training data using...
 - a standard supervised objective
 - backpropagation to compute gradients
 - your favorite optimizer (e.g. Adam)
- **Pro:** fits into the standard ML recipe
- **Pro:** still works if N is large
- **Con:** backpropagation requires $\sim 3x$ the memory and computation time as the forward computation
- **Con:** you might not have access to the model weights at all (e.g. because the model is proprietary)

Option B: In-context learning

- **Definition:**
 - feed training examples to the LLM as a prompt
 - allow the LLM to infer patterns in the training examples during inference (i.e. decoding)
 - take the output of the LLM following the prompt as its prediction
- **Con:** the prompt may be very long and Transformer LMs require $O(N^2)$ time/space where N = length of context
- **Pro:** no backpropagation required and only one pass through the training data
- **Pro:** does not require model weights, only API access

In-context learning

- ❑ In-context learning was popularized in the original GPT-3 paper as a way to use language models to learn tasks given only a few examples.
- ❑ On many benchmark NLP benchmarks, in-context learning is competitive with models trained with much more labeled data.
- ❑ **Few-shot prompting** can be used as a technique to enable **in-context learning** where we provide demonstrations in the prompt to steer the model to better performance.



In-context learning

- LLMs can solve novel downstream tasks by just conditioning on a few demonstrations of the task in its prefix.

Circulation revenue has increased by 5% in Finland. // Positive

Panostaja did not disclose the purchase price. // Neutral

Paying off the national debt will be extremely painful. // Negative

The company anticipated its operating profit to improve. // _____

LM

Circulation revenue has increased by 5% in Finland. // Finance

They defeated ... in the NFC Championship Game. // Sports

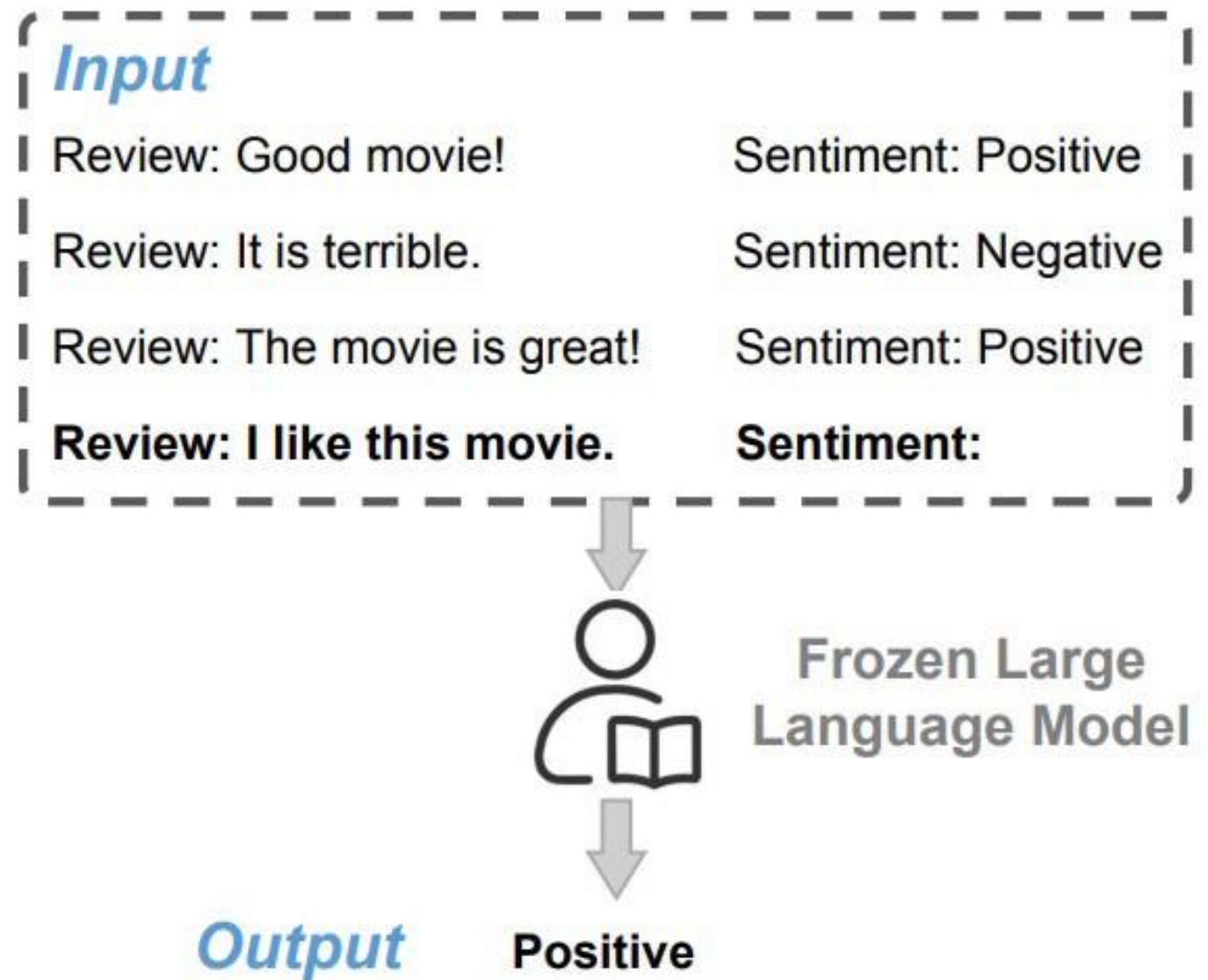
Apple ... development of in-house chips. // Tech

The company anticipated its operating profit to improve. // _____

LM

Few-shot In-context Learning

- ❑ Few-shot learning can be done via in-context learning
- ❑ Typically, a task description is presented first
- ❑ Then a sequence of input/output pairs from a training dataset are presented in sequence



Few-shot In-context Learning (cont'd)

- ❑ Few-shot learning can be done via in- context learning
- ❑ Typically, a task description is presented first
- ❑ Then a sequence of input/output pairs from a training dataset are presented in sequence

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Why is in-context learning surprising?

- ❑ Unlike conventional machine learning: No optimization of any parameters.
 - This isn't unique: Meta-learning methods have trained models that learn from examples.
- ❑ The mystery is that the LM isn't trained to learn from examples.
- ❑ There's seemingly a mismatch between pretraining (what it's trained to do, which is next token prediction) and in-context learning (what we're asking it to do).
- ❑ This seems like magic!

How does this work?

- ❑ Are “**novel**” tasks really novel? Or are they buried somewhere within the trillions of words in the Common Crawl?
- ❑ If not, how can the model “**learn**” to solve a new task without any training (i.e., gradient updates to optimize model performance on the new task)?
- ❑ In-context learning can be sensitive to...
 - the order the training examples are presented
 - the balance of labels (e.g. positive vs. negative)
 - the number of unique labels covered
 - ... but is it actually?

What's in a demonstration?

Demonstrations

Distribution of inputs

Label space

*Format
(The use
of pairs)*

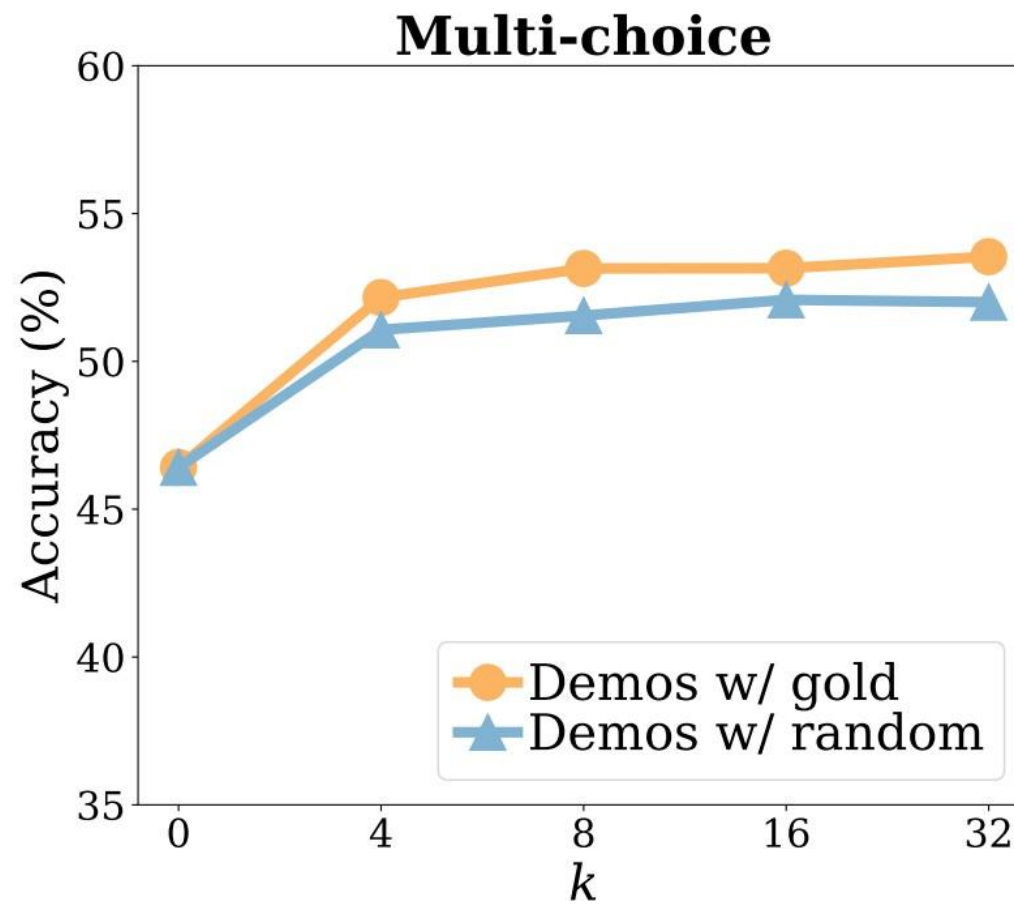
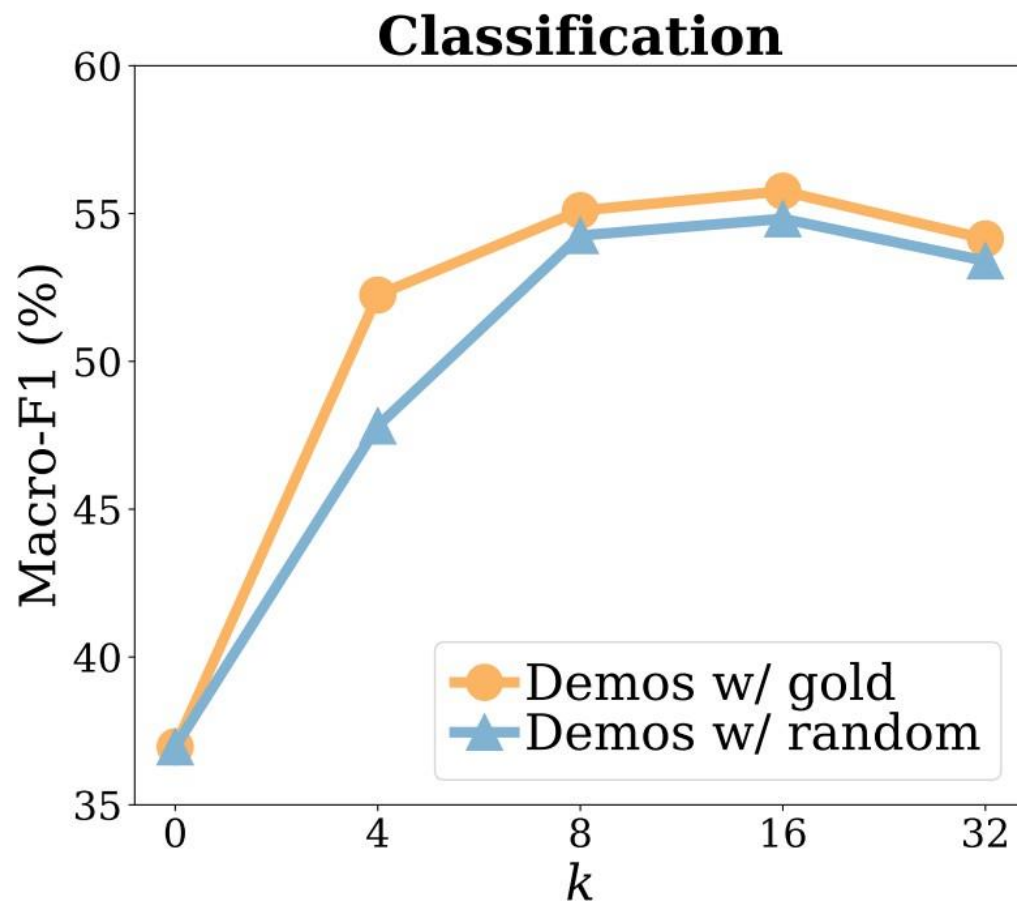
Circulation revenue has increased by 5% in Finland.	\n	Positive
Panostaja did not disclose the purchase price.	\n	Neutral
Paying off the national debt will be extremely painful.	\n	Negative

Test example

Input-label mapping

The acquisition will have an immediate positive impact.	\n	?
---	----	---

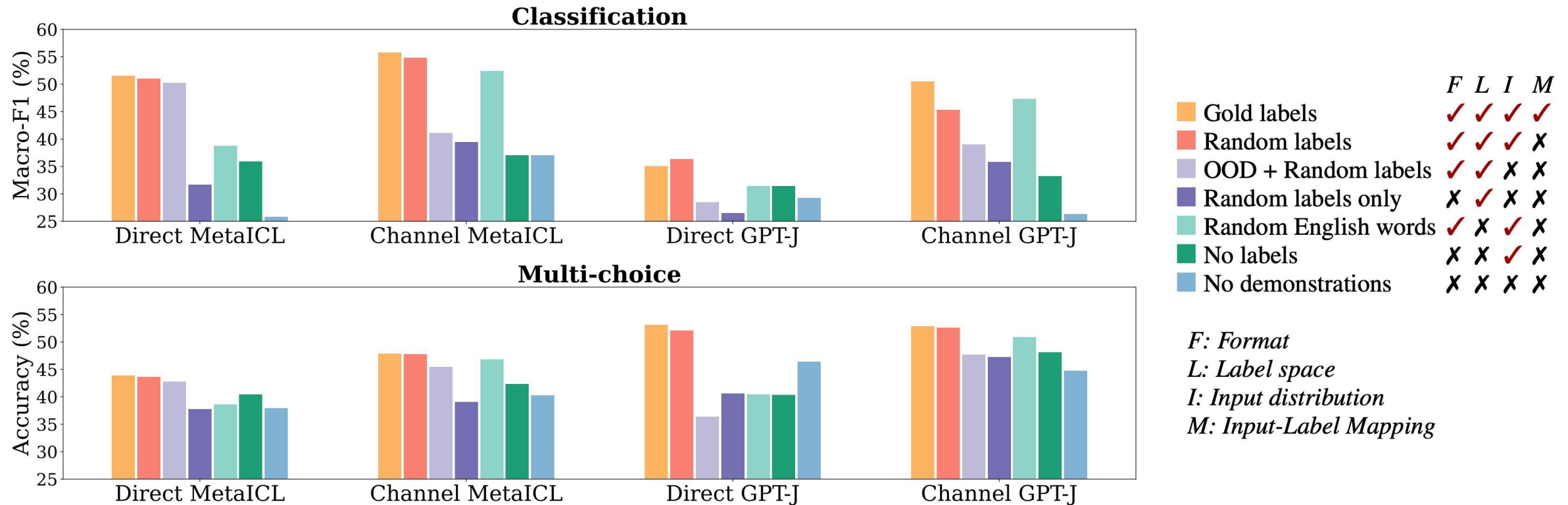
How important is input-label mapping?



What about other aspects of the demonstrations?

<i>Demos w/ gold labels</i>	(Format ✓ Input distribution ✓ Label space ✓ Input-label mapping ✓) Circulation revenue has increased by 5% in Finland and 4% in Sweden in 2008. \n positive Panostaja did not disclose the purchase price. \n neutral
<i>Demos w/ random labels</i>	(Format ✓ Input distribution ✓ Label space ✓ Input-label mapping ✗) Circulation revenue has increased by 5% in Finland and 4% in Sweden in 2008. \n neutral Panostaja did not disclose the purchase price. \n negative
<i>OOD Demos w/ random labels</i>	(Format ✓ Input distribution ✗ Label space ✓ Input-label mapping ✗) Colour-printed lithograph. Very good condition. Image size: 15 x 23 1/2 inches. \n neutral Many accompanying marketing claims of cannabis products are often well-meaning. \n negative
<i>Demos w/ random English words</i>	(Format ✓ Input distribution ✓ Label space ✗ Input-label mapping ✗) Circulation revenue has increased by 5% in Finland and 4% in Sweden in 2008. \n unanimity Panostaja did not disclose the purchase price. \n wave
<i>Demos w/o labels</i>	(Format ✗ Input distribution ✓ Label space ✗ Input-label mapping ✗) Circulation revenue has increased by 5% in Finland and 4% in Sweden in 2008. Panostaja did not disclose the purchase price.
<i>Demos labels only</i>	(Format ✗ Input distribution ✗ Label space ✓ Input-label mapping ✗) positive neutral

What about other aspects of the demonstrations?



Justification

- ❑ In-context learning can be thought of as “**locating**” **latent concepts** the LM has acquired from pretraining data.
- ❑ All components of the prompt (inputs, outputs, formatting, and the input-output mapping) can provide information for inferring the latent concept.

Circulation revenue has increased by 5% in Finland. // Positive

Panostaja did not disclose the purchase price. // Neutral

Paying off the national debt will be extremely painful. // Negative

The company anticipated its operating profit to improve. // _____

LM

Circulation revenue has increased by 5% in Finland. // Finance

They defeated ... in the NFC Championship Game. // Sports

Apple ... development of in-house chips. // Tech

The company anticipated its operating profit to improve. // _____

LM

This is a nice study, but it is also limited

- ❑ Only studied simple classification tasks
- ❑ The effect of these different aspects of demonstrations on generation tasks is unclear

Follow up work tells a different story

- ❑ Yoo et al. (2022) shows that input-label mappings matter quite significantly when using different experimental conditions and eval metrics.
- ❑ Madaan and Yazdanbakhsh (2022) show that random rationales degrade chain-of-thought performance, but other modifications to the rationale (e.g., wrong equations) don't affect it too much.

How does the LLM make use of (any aspect of) the demonstration?

- “We investigate the hypothesis that transformer-based in-context learners implement standard learning algorithms implicitly, by encoding smaller models in their activations, and updating these implicit models as new examples appear in the context.” — Akyürek et al., ICLR 2023

Can a Transformer do linear regression?

- Akyürek et al., ICLR 2023: theoretically proved that LLMs can learn a small linear model:

$$\sum_i \mathcal{L}(\mathbf{w}^\top \mathbf{x}_i, y_i) + \lambda \|\mathbf{w}\|_2^2$$

- The above linear regression loss has a *closed form* solution:

$$\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

- But can also be solved via gradient descent

$$\mathbf{w}' = \mathbf{w} - \alpha \frac{\partial}{\partial \mathbf{w}} \left(\mathcal{L}(\mathbf{w}^\top \mathbf{x}_i, y_i) + \lambda \|\mathbf{w}\|_2^2 \right) = \mathbf{w} - 2\alpha(\mathbf{x}\mathbf{w}^\top \mathbf{x} - y\mathbf{x} + \lambda\mathbf{w})$$

□ Transformers can implement some primitive operations useful for learning

It will be useful to first establish a few computational primitives with simple transformer implementations. Consider the following four functions from $\mathbb{R}^{H \times T} \rightarrow \mathbb{R}^{H \times T}$:

mov($H; s, t, i, j, i', j'$): selects the entries of the s^{th} column of H between rows i and j , and copies them into the t^{th} column ($t \geq s$) of H between rows i' and j' , yielding the matrix:

$$\left[\begin{array}{c|cc} & H_{:,i-1,t} & \\ H_{:,t} & H_{i':j',s} & H_{:,t+1:} \\ & H_{j,t} & \end{array} \right].$$

mul($H; a, b, c, (i, j), (i', j'), (i'', j'')$): in *each* column $\mathbf{h}$ of H , interprets the entries between i and j as an $a \times b$ matrix A_1 , and the entries between i' and j' as a $b \times c$ matrix A_2 , multiplies these matrices together, and stores the result between rows i'' and j'' , yielding a matrix in which each column has the form $[\mathbf{h}_{:i''-1}, A_1 A_2, \mathbf{h}_{j'':}]^\top$.

div($H; (i, j), i', (i'', j'')$): in each column $\mathbf{h}$ of H , divides the entries between i and j by the absolute value of the entry at i' , and stores the result between rows i'' and j'' , yielding a matrix in which every column has the form $[\mathbf{h}_{:i''-1}, \mathbf{h}_{i:j}/|\mathbf{h}_{i'}|, \mathbf{h}_{j'':}]^\top$.

aff($H; (i, j), (i', j'), (i'', j''), W_1, W_2, b$): in each column $\mathbf{h}$ of H , applies an affine transformation to the entries between i and j and i' and j' , then stores the result between rows i'' and j'' , yielding a matrix in which every column has the form $[\mathbf{h}_{:i''-1}, W_1 \mathbf{h}_{i:j} + W_2 \mathbf{h}_{i':j'} + b, \mathbf{h}_{j'':}]^\top$.

Lemma 1. *Each of mov, mul, div and aff can be implemented by a single transformer decoder layer: in Eq. (1) and Eq. (4), there exist matrices W^Q, W^K, W^V, W^F, W_1 and W_2 such that, given a matrix H as input, the layer's output has the form of the corresponding function output above.*¹

-
- Using these primitives, they show that Transformers can implement one step of gradient descent

Theorem 1. *A transformer can compute Eq. (11) (i.e. the prediction resulting from single step of gradient descent on an in-context example) with constant number of layers and $O(d)$ hidden space, where d is the problem dimension of the input x . Specifically, there exist transformer parameters θ such that, given an input matrix of the form:*

$$H^{(0)} = \begin{bmatrix} \cdots & 0 & y_i & 0 & \cdots \\ & \mathbf{x}_i & 0 & \mathbf{x}_n & \end{bmatrix}, \quad (12)$$

the transformer's output matrix $H^{(L)}$ contains an entry equal to $\mathbf{w}'^\top \mathbf{x}_n$ (Eq. (11)) at the column index where $\mathbf{x}_n$ is input.

Does ICL encode meaningful intermediate quantities?

- Probe networks show that Transformers encode useful intermediate computations while training the linear models.

To do so, we identify two intermediate quantities that we expect to be computed by gradient descent and ridge-regression variants: the **moment vector** $X^\top Y$ and the (min-norm) least-square estimated **weight vector** w_{OLS} , each calculated after feeding n exemplars. We take a trained in-context learner, freeze its weights, then train an auxiliary **probing** model (Alain & Bengio, 2016) to attempt to recover the target quantities from the learner's hidden representations. Specifically, the probe model takes hidden states at a layer $H^{(l)}$ as input, then outputs the prediction for target variable. We define a probe with **position-attention** that computes (Appendix E):

$$\alpha = \text{softmax}(\mathbf{s}_v) \tag{20}$$

$$\hat{\mathbf{v}} = \text{FF}_v(\alpha^\top W_v H^{(l)}) \tag{21}$$